

Техническая документация проекта «Финни»

Оглавление

Введение	1
Назначение документа	1
Краткое описание продукта	1
Требования к окружению и сборка	2
Требования к окружению (ОС, Node.js, инструменты).....	2
Версии ключевых инструментов и зависимостей	3
Пошаговая инструкция сборки APK-файла релизной версии.....	3
Архитектура решения	4
Функциональная архитектура (основные сценарии пользователя).....	4
Компонентная архитектура (модули, экраны, сторы)	5
Схема развёртывания и спецификация OpenAPI	6
Структура данных.....	7
Профиль пользователя	7
Игровая экономика (монеты, ресурсы).....	7
Задания и уроки	8
Прогресс пользователя.....	9
Матрица соответствия функциональным требованиям	9
Таблица: требование → статус реализации → ссылка на экран/модуль/тест.....	10
Формулы и правила игровой экономики	11
Расчёт баланса и покупки	11
Планирование бюджета.....	11
Накопления и финансовая цель.....	12
Награды, точность и уровень	12
Состояние и рост питомца	12
Карта образовательного контента	13
Таблица: тема → ожидаемый навык → сценарий → правильная логика → объяснение для ребёнка	13
UX/UI-решения	14
Обоснование ключевых интерфейсных решений	14
Настройки доступности	14
Разрешения и данные Android.....	15
Перечень запрашиваемых разрешений.....	15

Состав собираемых данных.....	15
Процедура удаления профиля/данных	15
Тестирование.....	16
Набор тест-кейсов.....	16
Краткий отчёт о проверке на физическом устройстве	17
Известные ограничения и план развития.....	17
Текущие ограничения прототипа	17
План дальнейшего развития	18
Сторонние библиотеки и лицензии.....	18
Библиотеки и пакеты	18
Шрифты, изображения, звуки — источники и лицензии	19

1. Введение

1.1. Назначение документа

Настоящий документ содержит техническую документацию мобильного приложения «Финни», разработанного в рамках хакатона «Лидеры цифровой трансформации» (ЛЦТ). Документ описывает архитектуру решения, требования к окружению и порядок сборки релизной версии приложения, структуру данных, реализованный функционал, используемые сторонние компоненты, а также результаты тестирования. Документ предназначен для членов жюри, экспертов и технических специалистов, оценивающих проект, и может использоваться как справочный материал при дальнейшей разработке и сопровождении продукта.

1.2. Краткое описание продукта

«Финни» — мобильное приложение для развития финансовой грамотности у детей 7–11 лет, реализованное в формате игры с виртуальным питомцем. Бытовые финансовые решения (кормление питомца, выбор одежды, накопление на цель) обёрнуты в понятный ребёнку игровой цикл, через который в интерактивной форме подаются базовые понятия: обязательные и необязательные траты, планирование бюджета, накопление на цель. Обучающий контент организован в виде последовательных глав и уроков (теория, задания, повторение), которые дополняют игровой процесс.

Приложение реализовано на React Native (Expo, TypeScript), работает полностью офлайн — без серверной части и подключения к интернету, весь прогресс пользователя хранится локально на устройстве.

2. Требования к окружению и сборка

2.1. Требования к окружению (ОС, Node.js, инструменты)

Для разработки и сборки приложения используются следующие компоненты окружения:

- Операционная система: Windows (сборка выполнялась в терминале Git Bash / MINGW64); процесс сборки не зависит от ОС — поддерживается также macOS и Linux.
- Node.js: v22.19.0 (LTS-совместимая версия, актуальная на момент разработки).
- Пакетный менеджер: npm.
- Инструмент сборки: EAS CLI (eas-cli), запускается через `npm eas-cli` без обязательной глобальной установки.
- Фреймворк: Expo (managed workflow), маршрутизация — Expo Router.

- Учётная запись Expo (EAS) с правами на проект для запуска облачной сборки.
- Для локальной сборки под Android дополнительно требуются Android Studio, Android SDK и JDK (используется командой при сборке через Android Studio как альтернативный способ, описанный в п. 2.3).

2.2. Версии ключевых инструментов и зависимостей

Компонент	Версия
Node.js	22.19.0
npm	согласно установленному Node.js
TypeScript	6.0.3
Expo SDK	57.0.26
React	19.2.3
React DOM	19.2.3
React Native	0.86.3
Expo Router	57.0.24
Zustand	5.0.15
@react-native-async-storage/async-storage	2.2.0
react-native-reanimated	4.5.1
react-native-worklets	0.10.1
react-native-gesture-handler	2.32.0
react-native-screens	4.26.0
react-native-safe-area-context	5.7.0
react-native-svg	15.15.4
react-native-keyboard-controller	1.21.9
EAS CLI	24.8.0 (актуальная на момент сборки)

Полный перечень зависимостей и их версий приведён в файле `package.json` в корне репозитория.

2.3. Пошаговая инструкция сборки APK-файла релизной версии

Сборка выполняется через облачный сервис EAS Build (Expo Application Services), профиль `preview` (тип сборки — APK, внутреннее распространение).

Установить зависимости проекта:

```
npm install
```

Авторизоваться в EAS CLI под аккаунтом, имеющим права на проект:

```
npx eas-cli login
```

Проверить текущего пользователя можно командой `npx eas-cli whoami`.

Убедиться, что в `app.json` поле `extra.eas.projectId` и поле `slug` соответствуют проекту, привязанному к используемому аккаунту на `expo.dev` (несовпадение `slug` с проектом по `projectId` приводит к ошибке сборки).

Запустить сборку Android-версии с профилем preview:

```
npx eas-cli build --platform android --profile preview
```

Сборка ставится в очередь на серверах EAS (бесплатный тариф — общая очередь, время ожидания может составлять более часа) и выполняется удалённо; прогресс отображается в терминале и на странице сборки на expo.dev.

После завершения сборки APK-файл доступен для скачивания по ссылке в терминале или в разделе Builds на expo.dev.

Альтернативный способ (локальная сборка): сборка Android-приложения может быть выполнена локально через Android Studio при наличии установленных Android SDK и JDK, командой `npm run android` (сборка debug-версии) либо через штатные средства Android Studio для генерации подписанного релизного APK/AAB.

3. Архитектура решения

3.1. Функциональная архитектура (основные сценарии пользователя)

Приложение построено вокруг единого игрового цикла с виртуальным питомцем и делится на следующие основные сценарии:

Онбординг (первый запуск). Приветственный экран → ввод имени питомца → выбор цвета питомца → вылупление питомца из яйца → знакомство с питомцем. После онбординга пользователь попадает в основное приложение, где дальнейшее знакомство с функциями идёт через пошаговый обучающий сценарий (единое состояние `tutorialStage`, управляющее очередностью подсказок): получение ежедневной награды → кормление питомца → создание первой финансовой цели → открытие раздела уроков → получение предмета гардероба.

Основной цикл приложения реализован через нижнюю панель навигации из пяти разделов:

Учёба — карта глав и уроков (теория, задания, повторение), проходятся последовательно; статус урока открыт/заблокирован зависит от прогресса.

Цели — создание и редактирование финансовых целей-копилок, детальный экран цели, история операций, экран планирования бюджета.

Дом — центральный экран с питомцем, ежедневная награда, диалоги с питомцем.

Еда — магазин еды/кухня, кормление питомца.

Гардероб (Магазин) — покупка и надевание предметов одежды на питомца.

Дополнительные сценарии: прохождение отдельного урока (`lesson/[id]`) с завершающим экраном результата, завершение главы, экран прогресса пользователя, серии

ежедневных входов (streak), профиль пользователя, родительский экран (контроль/настройки для родителя), глоссарий терминов, общие настройки приложения.

3.2. Компонентная архитектура (модули, экраны, сторы)

Проект построен на React Native + Expo Router, структура кода разделена по зонам ответственности:

app/ — файловый роутинг (Expo Router): корневые экраны (бюджет, цели, история, профиль, настройки и т.д.), группа (onboarding) — экраны первого запуска, группа (tabs) — пять основных разделов нижней навигации со своим _layout.tsx, и lesson/[id] — динамический маршрут прохождения урока.

game/ — ядро игровой логики:

store/gameStore.ts — центральное состояние игры (прогресс, экономика, цели, обучение, стадия туториала);

services/ — сервисные модули (dialogService — показ диалогов питомца, tutorialHintStore — управление подсказками);

components/ — игровые UI-компоненты (питомец и его анимации — PetSprite, PetWithHat, PetSpeechBubble; элементы интерфейса игры — GameHud, HungerMeter, CustomTabBar, DailyRewardModal, TasksModal, TutorialHand/TutorialPointer, AnimatedNumber и др.);

catalog.ts, wardrobe.ts, purchaseRules.ts — игровые данные и правила (каталог предметов, правила покупок).

store/profileStore.ts — отдельное хранилище профиля пользователя (имя и цвет питомца, персональные данные), независимое от игрового состояния.

features/ — предметная логика, вынесенная из экранов:

learning/ — образовательный контент (content.ts — главы и уроки, lessonFlows.ts — сценарии прохождения уроков);

goals/catalog.ts — каталог целей;

budget/BudgetControls.tsx — компоненты управления бюджетом.

content/ — статические справочные данные (стадии вылупления питомца, доступные цвета, ассеты питомца), не зависящие от игровой логики.

ui/ — общая дизайн-система приложения: базовые компоненты (AppText, Button, TextField, Screen, BackgroundScreen, PetStage) и файл темы (theme.ts).

components/ — переиспользуемые компоненты общего назначения (FinnyButton).

assets/ — медиаресурсы, структурированные по смысловым разделам: персонажи питомца (по стадиям и цветам), еда, гардероб, обучающий контент, элементы интерфейса.

Такое разделение (состояние — в сторах, контент обучения и каталоги — в отдельных модулях, интерфейс — в ui/) позволяет добавлять новые уроки, предметы или задания без изменения основной логики экранов.

3.3. Схема развёртывания и спецификация OpenAPI

Пункт неприменим: приложение не имеет серверной части. Все данные (профиль, прогресс, игровая экономика) хранятся локально на устройстве пользователя (Zustand + AsyncStorage), сетевые запросы к бэкенду отсутствуют, приложение работает полностью офлайн. Схема развёртывания и спецификация OpenAPI не разрабатывались в связи с отсутствием клиент-серверного взаимодействия.

4. Структура данных

4.1. Профиль пользователя

Профиль хранится в отдельном сторе profileStore (Zustand + persist в AsyncStorage, ключ finny-profile, версия схемы — 1) и не зависит от игрового прогресса:

Поле	Тип	Назначение
playerName	string	Имя ребёнка
petName	string	Имя питомца, заданное при онбординге
petColorId	PetColorId ""	Выбранный цвет питомца
onboardingDone	boolean	Пройден ли первичный онбординг
tutorialFeedDone	boolean	Пройден ли обучающий шаг «первое кормление»
notificationsEnabled	boolean	Включены ли уведомления (по умолчанию true)
soundEnabled	boolean	Включён ли звук (по умолчанию false)

Профиль не содержит персональных данных сверх имени ребёнка и имени питомца — email, телефон, геолокация и т. п. не собираются и не хранятся.

4.2. Игровая экономика (монеты, ресурсы)

Экономика реализована в основном сторе `gameStore` (`persist` в `AsyncStorage`, ключ `finny-game`, текущая версия схемы — 6, с функцией миграции для совместимости со старыми сохранениями).

Баланс и ресурсы:

Поле	Тип	Назначение
<code>coins</code>	<code>number</code>	Текущий баланс монет
<code>xp / level</code>	<code>number</code>	Опыт и уровень (уровень = $\lfloor \text{xp} / 100 \rfloor + 1$)
<code>hunger</code>	<code>number (0–100)</code>	Сытость питомца
<code>piggy</code>	<code>number</code>	Сумма в копилке (для накопления на цели)
<code>inventory</code>	<code>Partial<Record<FoodId, number>></code>	Купленная, но ещё не съеденная еда, по видам
<code>budgetPlan</code>	<code>{ need, want, save }</code>	Распределение бюджета в процентах (сумма = 100)
<code>transactions</code>	<code>MoneyTransaction[]</code>	История операций (хранятся последние 100 записей)

Каталог еды (`game/catalog.ts`) — 32 позиции в 6 категориях (фрукты, овощи, готовая еда, сладкое, витамины, напитки), каждая позиция описана как `{ id, name, category, price, nutrition, image }`; цена — от 5 до 20 монет, показатель `nutrition` определяет, насколько кормление повышает сытость питомца.

Цели-копилки (`features/goals/catalog.ts`) — 6 шаблонов целей `{ id, name, target, image }` со стоимостью от 15 до 150 монет (например, «Колпак в точку» — 15, «Велосипед» — 120, «Ноутбук» — 150). Активная цель пользователя хранится как `GoalRecord: { id, templateId, name, target, saved, createdAt, purchasedAt? }`.

Гардероб — предметы одежды (`HatId`, каталог в `game/wardrobe.ts`), в сторе хранятся `equippedHat` (надетый предмет) и `ownedHats` (список купленных).

- 4.3. Задания и уроки**
- Образовательный контент вынесен в отдельный модуль `features/learning/content.ts`, полностью независимый от игровой логики.
- Иерархия: 3 главы (`ChapterId: budget, savings, payments`) → уроки внутри главы (11 уроков суммарно, по 3–4 на главу).

Структура главы: { id, title, subtitle, color, lessons: string[] }.

Структура урока (LessonDefinition):

Поле	Назначение
id, chapter, number	Идентификатор, принадлежность к главе, порядковый номер
type	Тип: theory (теория), test (задание), repetition (повторение)
kind	Конкретный сценарий/механика урока (9 видов: budget-select, priority, trip, truth, goal, weekly-save, sort-coins, purchase-balance, coinpay-change)
title, kicker	Заголовок и подпись («Глава 1, Урок 1»)
introTitle, introText	Вводный экран урока
theory	Массив карточек теории: { title, text, image? }
image	Иллюстрация урока

Прогресс по урокам хранится в gameStore: completedLessons (пройденные уроки), claimedLessonRewards (забранные награды за уроки), lessonAccuracy (процент правильных ответов по каждому уроку), а также аналогично на уровне глав — completedChapters, claimedChapterRewards. Открытие следующего урока/главы зависит от прохождения предыдущего элемента по порядку (см. п. 3.1 функциональной архитектуры).

4.4. Прогресс пользователя

Прогресс складывается из нескольких независимых, но связанных срезов состояния в gameStore:

- **Обучающий сценарий (онбординг):** tutorialStage — число от 0 до 8, определяющее, какой шаг знакомства с приложением сейчас активен (см. функциональную архитектуру, п. 3.1).

- **Учёба:** completedLessons, completedChapters, lessonAccuracy, claimedLessonRewards, claimedChapterRewards.

- **Ежедневная активность:** streakDays (серия дней подряд), lastRewardDate (дата последнего получения ежедневной награды), dailyActivity — { date, fed, purchased } (сколько раз покормили питомца и сколько покупок сделали за текущий день).

- **Разовые события:** taskRewardsClaimed (полученные награды за отдельные задания), milestonesSeen (просмотренные важные подсказки/уведомления, чтобы не показывать их повторно).

– **Общий рост:** `xp` и `level`, растущие за прохождение уроков и глав (30 монет + 25 опыта за урок/главу).

Все перечисленные данные (4.1–3.4) хранятся локально на устройстве через `zustand/middleware persist` с адаптером `AsyncStorage`, без синхронизации с сервером. Для игрового стора предусмотрена версионированная миграция схемы (`migrate`), которая при обновлении версии приложения аккуратно приводит ранее сохранённые данные пользователя к новой структуре, не сбрасывая прогресс.

5. Матрица соответствия функциональным требованиям

Матрица составлена по обязательным требованиям технического задания и по фактически описанной архитектуре проекта. Статус «частично» означает, что основная механика реализована, но отдельная деталь требования не подтверждена предоставленной документацией и должна быть проверена в финальной сборке.

5.1. Таблица: требование → статус реализации → ссылка на экран/модуль/тест

Требование	Статус	Что реализовано / комментарий	Экран, модуль или тест
2.5.1. Первый запуск и локальный профиль	Реализовано	Онбординг без регистрации; имя ребёнка/питомца и цвет хранятся локально; <code>tutorialStage</code> и <code>tutorialHintStore</code> поддерживают пошаговые подсказки.	<code>app/(onboarding)/*</code> ; <code>store/profileStore.ts</code> ; <code>game/services/tutorialHintStore</code>
2.5.2. Создание виртуального питомца	Реализовано	Ввод имени, выбор цвета, сценарий вылупления и знакомство с питомцем.	<code>app/(onboarding)/*</code> ; <code>content/</code> ; <code>store/profileStore.ts</code>

2.5.3. Главный экран	Частично	Домашний экран содержит питомца, баланс/состояние и быстрый доступ к основным разделам. Одновременное отображение текущей цели, накоплений и активного задания требует финальной визуальной проверки.	app/(tabs)/*; GameHud; HungerMeter; CustomTabBar
2.5.4. Игровая валюта и получение дохода	Реализовано	Внутриигровые coins; награды за уроки/главы; ежедневная награда; история операций.	game/store/gameStore.ts; DailyRewardModal; lesson/[id]
2.5.5. Планирование бюджета	Частично	budgetPlan разделён на need/want/save, сумма долей контролируется; есть экран планирования бюджета. Сравнение плана с фактическими расходами после подтверждения в документации не описано.	features/budget/BudgetControls.tsx; game/store/gameStore.ts
2.5.6. Покупки и расходы	Реализовано	Каталог еды и гардероба, цены, списание средств, инвентарь, purchaseRules и журнал transactions. Покупка при недостатке средств должна блокироваться правилом баланса.	game/catalog.ts; game/wardrobe.ts; game/purchaseRules.ts; game/store/gameStore.ts

2.5.7. Накопления и финансовая цель	Частично	6 шаблонов целей; target/saved; создание и редактирование цели, отображение прогресса. Отдельное подтверждение снятия средств с цели документально не подтверждено.	features/goals/catalog.ts; экраны целей; game/store/gameStore.ts
2.5.8. Финансовые задания	Реализовано	3 темы: бюджет, сбережения, платежи и покупки; 11 уроков; 9 игровых механик; объяснения при правильном и ошибочном ответе.	features/learning/content.ts; features/learning/lessonFlows.ts; lesson/[id]
2.5.9. Последствия и обратная связь	Реализовано	Изменение баланса, накоплений и сытости отражается в состоянии; ошибки сопровождаются пояснением; используются диалоги питомца и экраны результата.	game/services/dialogService; gameStore; уроки и экраны результата
2.5.10. Прогресс и развитие питомца	Частично	Есть hunger, xp, level, серии входов и визуальные стадии вылупления. Отдельная модель минимум из 3 стадий роста, зависящая от совокупности финансовых решений нескольких периодов, в доступном описании не подтверждена.	game/store/gameStore.ts; content/; PetSprite/PetWithHat
2.5.11. История и учебный прогресс	Реализовано	completedLessons/completedChapters, lessonAccuracy, история операций, экран прогресса и глоссарий.	game/store/gameStore.ts; экран истории; экран прогресса; глоссарий

2.5.12. Раздел для взрослого	Частично	Родительский экран и настройки предусмотрены архитектурой. Барьер взрослого и фактическая кнопка удаления/сброса должны быть подтверждены в релизной сборке.	родительский экран; настройки; profileStore/gameStore
2.5.13. Сохранение состояния и демонстрационный режим	Частично	Профиль и игровой прогресс сохраняются через Zustand persist + AsyncStorage. Отдельный демонстрационный режим с тестовым профилем и мгновенным переходом между периодами в документации не подтверждён.	finny-profile; finny-game; migrate
2.5.14. Управление контентом	Реализовано	Учебный контент вынесен в независимые content.ts и lessonFlows.ts; добавление уроков не требует изменения базовой навигации и стор.	features/learning/content.ts; features/learning/lessonFlows.ts

6. Формулы и правила игровой экономики

Ниже зафиксированы правила, которые следуют из структуры gameStore, каталогов и образовательных сценариев. Все денежные операции выполняются только во внутриигровой валюте; отрицательный баланс не допускается.

6.1. Расчёт баланса и покупки

Проверка возможности покупки. Покупка разрешена только при наличии достаточного количества монет.

$$purchaseAllowed = (coins \geq price)$$

Баланс после покупки. При успешной покупке стоимость списывается один раз. Если $coins < price$, состояние баланса и инвентаря не меняется.

$$coins_after = coins_before - price$$

Количество товара в инвентаре. Для еды купленная позиция помещается в inventory; для гардероба идентификатор добавляется в ownedHats.

$$inventory[item]_{after} = inventory[item]_{before} + 1$$

Каждая денежная операция должна добавляться в transactions с типом операции, суммой и служебными данными. В состоянии хранятся последние 100 записей истории.

6.2. Планирование бюджета

Сумма долей бюджета. План распределяется по трём направлениям: обязательные расходы, желания и накопления.

$$need + want + save = 100 \%$$

Сумма по категории. Процентная доля переводится в понятное ребёнку количество монет; итоговое распределение не должно превышать доступный баланс.

$$amount_category = round(coins \times share_category / 100)$$

Остаток. При корректном плане остаток неотрицателен. До подтверждения план можно корректировать.

$$remainder = coins - need_amount - want_amount - save_amount$$

6.3. Накопления и финансовая цель

Осталось накопить. target — стоимость цели, saved — уже накопленная сумма.

$$remaining = \max(target - saved, 0)$$

Прогресс цели. Используется для визуального индикатора прогресса; при $saved \geq target$ цель считается достигнутой.

$$progress = \min(100, saved / target \times 100 \%)$$

Регулярное пополнение. Формула используется в учебном сценарии сбережений: сколько нужно откладывать за период, чтобы достичь цели за заданное число недель.

$$weeklySave = \text{ceil}((target - saved) / weeks)$$

При переводе монет в накопления сумма перевода не может быть отрицательной и не может превышать свободный баланс. После операции уменьшается доступный баланс и увеличивается накопленная сумма цели/копилки на ту же величину.

Добавлено примечание ([C1]): Этой формулы (перевод процентной доли бюджета в монеты) нет в gameStore.ts — там setBudgetPlan только нормализует need/want/save до суммы 100%, без умножения на coins. Похоже на предположение о логике UI (budget-plan.tsx/BudgetControls.tsx), которую мы не проверяли. Стоит подтвердить по этим файлам или явно пометить как предполагаемую реализацию.

Добавлено примечание ([C2]): По коду gameStore.ts накопления работают не по каждой цели отдельно: depositPiggy/withdrawPiggy изменяют единое поле piggy, а не goal.saved (поле saved при миграции всегда сбрасывается в 0 и ни одним действием стора не обновляется). Если экран цели показывает прогресс через goal.saved — это, вероятно, не соответствует факту; нужно свериться с goal-detail.tsx/goal-editor.tsx, как именно считается прогресс.

6.4. Награды, точность и уровень

Награда за учебный элемент. В текущей конфигурации документа за завершение урока/главы указана награда 30 монет и 25 опыта. Массивы `claimedLessonRewards` и `claimedChapterRewards` предотвращают повторное получение одной и той же награды.

$$\text{coins_after} = \text{coins_before} + 30; \text{xp_after} = \text{xp_before} + 25$$

Точность урока. Значение сохраняется в `lessonAccuracy` и выводится на завершающем экране урока.

$$\text{accuracy} = \text{round}(\text{correct} / \text{total} \times 100 \%)$$

Уровень пользователя. Каждые 100 очков опыта повышают уровень на единицу.

$$\text{level} = \text{floor}(\text{xp} / 100) + 1$$

Ежедневная активность хранится через `lastRewardDate`, `streakDays` и `dailyActivity`. Повторная выдача одной и той же ежедневной награды в рамках одной даты должна блокироваться; серия входов изменяется при переходе к новой дате.

6.5. Состояние и рост питомца

Сытость после кормления. `nutrition` берётся из `game/catalog.ts`; значение `hunger` ограничено диапазоном 0–100.

$$\text{hunger_after} = \min(100, \text{hunger_before} + \text{nutrition})$$

Инвентарь после кормления. Кормление возможно только при наличии выбранной еды в инвентаре.

$$\text{inventory}[\text{item}]_after = \max(0, \text{inventory}[\text{item}]_before - 1)$$

Эмоциональная обратная связь питомца связана с текущим состоянием и событиями через `PetSprite/PetSpeechBubble` и `dialogService`. Долгосрочный рост в доступной реализации выражается через `xp/level`; отдельная трёхстадийная модель развития по совокупности финансовых решений должна быть подтверждена или добавлена перед финальной сдачей.

7. Карта образовательного контента

Образовательный контент организован в три главы: планирование бюджета, формирование сбережений, платежи и покупки. Контент отделён от интерфейсной логики и задаётся в `features/learning/content.ts`, а конкретные игровые сценарии — в `lessonFlows.ts`.

Добавлено примечание ([C3]): В дереве проекта есть `content/hatchStages.ts`, а в `assets/library/characters` — папки `eggs` (стадии 1-5 + `burst`), `child` и `adult`; в `learn.tsx` явно написано 'После первой главы Финни повзрослеет'. Похоже, стадийный рост (минимум `egg`→`child`→`adult`) уже реализован и завязан на прохождение главы, а не только на `xp/level`, как здесь написано. Стоит свериться с `hatchStages.ts` и `PetStage.tsx`, прежде чем писать, что этого нет.

7.1. Таблица: тема → ожидаемый навык → сценарий → правильная логика

→ объяснение для ребёнка

Тема	Ожидаемый навык	Сценарий	Правильная логика	Объяснение для ребёнка
Планирование бюджета	Понимать, зачем нужен бюджет и почему денег может не хватить на всё сразу.	budget-select: распределение расходов на важное, желаемое и накопления.	Сначала закрыть обязательные расходы, затем выбрать желания и оставить часть суммы на цель.	«Сначала позаботься о важном, потом выбери то, что хочется, и не забудь немного отложить».
Планирование бюджета	Уметь расставлять расходы по приоритету.	priority: перетаскивание карточек расходов по важности.	Обязательные расходы и учебные/базовые потребности должны находиться выше необязательных желаний.	«Не все покупки одинаково срочные: тетрадь нужна сейчас, а сладость можно купить позже».
Планирование бюджета	Выбирать нужные покупки в условиях ограниченного бюджета и оставлять резерв.	trip: сбор Финни в поездку/поход из набора предметов.	Выбрать необходимое для безопасности и базовых потребностей; отказаться от лишнего, если оно уменьшает запас.	«Купи то, без чего поездка не получится, а часть монет оставь на неожиданности».
Планирование бюджета	Закрепить определения и причинно-следственные связи.	truth: карточки «верно/неверно».	Бюджет — это заранее составленный план распределения денег между расходами и накоплениями.	«Бюджет помогает заранее решить, сколько потратить, оставить и отложить».
Формирование сбережений	Понимать связь цели, её стоимости и уже накопленной суммы.	goal: выбор финансовой цели и просмотр прогресса.	Сравнить стоимость цели с накопленной суммой и определить, сколько ещё требуется.	«Цель становится ближе каждый раз, когда ты откладываешь часть монет».
Формирование сбережений	Рассчитывать регулярную сумму накопления.	weekly-save: расчёт суммы на неделю.	Вычесть уже накопленное из стоимости цели и разделить остаток на число периодов.	«Сначала узнай, сколько не хватает, а потом раздели эту сумму на оставшиеся недели».

Добавлено примечание ([C4]): Механика 'перетаскивание карточек' (drag-and-drop) не подтверждена content.ts — там только текстовое описание сценария ('Разложи покупки на «Сейчас» и «Позже»'), сама интерактивная реализация — в lessonFlows.ts, которого у нас нет. Формулировку лучше смягчить или подтвердить перед сдачей, чтобы не описывать несуществующий интерфейс.

Добавлено примечание ([C5]): content.ts описывает savings-1 как теоретический урок (type: theory) про разницу между мечтой и целью, с двумя карточками теории (велосипед в пример, шаги 'как превратить мечту в цель') — интерактивного выбора цели/прогресса в этом уроке по content.ts не видно. Механика 'выбор цели и просмотр прогресса', возможно, относится к экрану целей (goal-editor/goal-detail), а не к этому уроку. Файла lessonFlows.ts у нас нет — стоит свериться, прежде чем публиковать.

Формирование сбережений	Закрепить понятие накоплений и отказ от необязательной траты.	truth / repetition: карточки финансовых утверждений.	Накопления — это деньги, которые не тратятся сейчас и сохраняются для будущей цели.	«Если не потратить всё сегодня, завтра будет легче купить то, что действительно важно».
Платежи и покупки	Соотносить цену товара с доступным балансом.	purchase-balance: решение, можно ли совершить покупку и сколько останется.	Покупка возможна, если $\text{balance} \geq \text{price}$; остаток равен $\text{balance} - \text{price}$.	«Сначала посмотри цену и свой баланс. Если монет меньше, покупку лучше отложить».
Платежи и покупки	Понимать состав суммы из монет и выбирать достаточный набор.	sort-coins: сбор нужной суммы из монет.	Сумма выбранных монет должна точно соответствовать цене или условию задания.	«Собери нужную сумму: каждая монета добавляет часть стоимости».
Платежи и покупки	Считать сдачу и проверять корректность расчёта.	coinpay-change: оплата и определение сдачи.	$\text{change} = \text{payment} - \text{price}$ при $\text{payment} \geq \text{price}$.	«Сдача — это то, что возвращают после оплаты, если ты дал больше, чем стоит покупка».
Платежи и покупки	Закрепить правило проверки цены перед покупкой.	truth / repetition: карточки «верно/неверно».	Перед покупкой нужно проверить цену и сопоставить её с доступными средствами.	«Цена помогает понять, хватит ли монет на покупку».

Формулировки построены короткими фразами, а ошибки не наказывают потерей прогресса: после неправильного действия ребёнок получает объяснение и возможность попробовать снова.

8. UX/UI-решения

8.1. Обоснование ключевых интерфейсных решений

Решение	Почему оно выбрано
Нижняя навигация из пяти разделов	Основные сценарии доступны в один переход и имеют постоянное положение: Учёба, Цели, Дом, Еда, Гардероб.
Дом как центральный экран	Виртуальный питомец остаётся эмоциональным центром приложения; через его состояние ребёнок видит последствия решений.
Пошаговый tutorialStage	Новые функции раскрываются последовательно: награда → кормление → цель → уроки → гардероб. Это снижает когнитивную нагрузку первого запуска.

Короткие экраны уроков	Одна задача или один смысловой блок на экране; после действия сразу показывается результат и простое объяснение.
Единая дизайн-система ui/	AppText, Button, TextField, Screen, BackgroundScreen и PetStage обеспечивают одинаковые отступы, типографику и поведение элементов.
Визуальная обратная связь	AnimatedNumber, HungerMeter, модальные окна наград и реплики питомца делают изменение баланса/состояния заметным без перехода к сложным отчётам.
Безопасная ошибка	Ошибочный ответ сопровождается пояснением и повторной попыткой; ранее достигнутый прогресс не обнуляется.
Офлайн-работа	Основной цикл не зависит от качества сети; ребёнок не сталкивается с внезапными сетевыми ошибками и не теряет локальный прогресс.

8.2. Настройки доступности

Настройка/требование	Статус	Комментарий
Короткий и понятный текст	Реализовано	Тексты уроков и подсказок рассчитаны на возраст 7–11 лет; сложные финансовые понятия объясняются примерами.
Крупные интерактивные зоны, ориентир $\geq 48 \times 48$ dp	Требует финальной проверки	Компоненты кнопок централизованы в ui/, поэтому размер можно проверять и корректировать в одном месте.
Основной текст около/выше 16 sp	Требует финальной проверки	Нужно проверить theme.ts и все исключения на реальном устройстве.
Цвет не является единственным сигналом	В основном реализовано	Успех/ошибка дополнительно передаются текстом, состоянием кнопки и репликой питомца.
Единообразная навигация и кнопка «Назад»	Реализовано	Маршрутизация централизована через Expo Router и общие компоненты экранов.
Отключение звука	Реализовано	soundEnabled хранится в profileStore; по умолчанию звук выключен.
Отключение анимаций / reduced motion	Не подтверждено	Отдельная настройка в описанной структуре данных не указана; рекомендуется добавить перед публикацией.

Системное увеличение шрифта	Не подтверждено	Нужно проверить поведение AppText при увеличенном системном размере шрифта.
Подписи для экранного диктора	Не подтверждено	Нужен отдельный аудит accessibilityLabel/accessibilityRole у интерактивных компонентов.

9. Разрешения и данные Android

9.1. Перечень запрашиваемых разрешений

Обязательный игровой сценарий не требует камеры, микрофона, геолокации, контактов, Bluetooth или доступа к файлам пользователя. Поэтому приложение не должно запрашивать опасные Android-разрешения для этих функций.

Разрешение	Статус	Обоснование
CAMERA	Не требуется	Камера не используется.
RECORD_AUDIO	Не требуется	Запись звука не используется.
ACCESS_FINE_LOCATION / ACCESS_COARSE_LOCATION	Не требуется	Геолокация не используется.
READ_CONTACTS / WRITE_CONTACTS	Не требуется	Контакты не используются.
BLUETOOTH_*	Не требуется	Bluetooth не входит в сценарий.
READ/WRITE_EXTERNAL_STORAGE	Не требуется	Профиль и прогресс сохраняются во внутреннем хранилище приложения через AsyncStorage.
INTERNET	Может присутствовать как обычное разрешение сборки Exro	Основная логика приложения работает офлайн и не выполняет запросы к серверной части. Наличие permission следует проверить в итоговом AndroidManifest финального APK.
POST_NOTIFICATIONS (Android 13+)	Не подтверждено / только при системных уведомлениях	В profileStore есть настройка notificationsEnabled, но пакет и фактическая реализация системных уведомлений в предоставленном описании не указаны.

9.2. Состав собираемых данных

Группа данных	Состав	Цель	Хранение
Профиль	playerName, petName, petColorId, onboardingDone	Локальная персонализация и состояние онбординга	Только устройство, AsyncStorage: finny-profile
Настройки	notificationsEnabled, soundEnabled	Пользовательские настройки приложения	Только устройство, finny-profile
Игровая экономика	coins, xp, level, hunger, piggy, inventory, budgetPlan	Работа игрового цикла	Только устройство, finny-game
Цели и покупки	GoalRecord, ownedHats, equippedHat, transactions	Накопления, история и персонализация питомца	Только устройство, finny-game
Учебный прогресс	completedLessons, completedChapters, lessonAccuracy, claimed rewards	Открытие уроков и отображение прогресса	Только устройство, finny-game
Ежедневная активность	streakDays, lastRewardDate, dailyActivity	Серия входов и ежедневные задания	Только устройство, finny-game

Не собираются: номер телефона, e-mail, точная геолокация, контакты, фотографии/аудиозаписи, реальные платёжные реквизиты, банковские данные и сведения о семейном бюджете. Серверной передачи данных и синхронизации в текущей версии нет.

9.3. Процедура удаления профиля/данных

Удаление должно выполняться локально без обращения к разработчику. Для финальной сборки процедура должна быть привязана к разделу взрослого или настройкам и работать следующим образом:

- Пользователь открывает раздел взрослого/настройки и выбирает «Удалить профиль» или «Сбросить данные».
- Приложение показывает отдельное подтверждение, чтобы исключить случайное удаление ребёнком.
- После подтверждения сбрасываются состояния profileStore и gameStore и удаляются локальные записи AsyncStorage с ключами finny-profile и finny-game.
- Пользователь возвращается к первому экрану онбординга; баланс, покупки, цели, уроки и настройки не восстанавливаются автоматически.

- Так как серверной части нет, дополнительных запросов на удаление данных отправлять не требуется.

Фактическое наличие кнопки и взрослого барьера необходимо проверить в релизной сборке; архитектура хранения позволяет выполнить удаление полностью на устройстве.

10. Тестирование

10.1. Набор тест-кейсов

ID	Проверка	Шаги	Ожидаемый результат
ТС-01	Чистая установка	Запустить приложение после очистки данных.	Открывается онбординг, приложение не требует регистрации/сети.
ТС-02	Создание питомца	Ввести имя, выбрать цвет, завершить вылупление.	Имя и цвет отображаются в приложении и сохраняются в profileStore.
ТС-03	Сохранение профиля	Закрыть приложение после онбординга и открыть снова.	Онбординг не повторяется; выбранный питомец восстановлен.
ТС-04	Ежедневная награда	Получить награду и повторно открыть модальное окно в ту же дату.	Баланс увеличивается один раз; повторное начисление в тот же день блокируется.
ТС-05	Покупка при достаточном балансе	Выбрать товар с $price \leq coins$ и подтвердить.	Баланс уменьшается на price; товар появляется в inventory/ownedHats; операция попадает в историю.
ТС-06	Покупка при недостатке средств	Выбрать товар с $price > coins$.	Покупка не проводится, баланс не становится отрицательным, показывается понятное сообщение.
ТС-07	Кормление питомца	Купить еду, затем покормить питомца.	Количество еды уменьшается на 1; hunger увеличивается не выше 100.

ТС-08	План бюджета	Изменить need/want/save и подтвердить допустимый план.	Сумма долей равна 100%; приложение показывает корректное распределение.
ТС-09	Некорректный план бюджета	Попробовать задать сумму долей больше/меньше 100% или распределение выше доступного бюджета.	Подтверждение блокируется либо значения нормализуются по правилам интерфейса.
ТС-10	Создание финансовой цели	Выбрать шаблон цели и создать её.	Отображаются target, saved=0 и корректный прогресс.
ТС-11	Пополнение цели	Перевести допустимую сумму в накопления.	Свободный баланс уменьшается, saved/piggy увеличивается на ту же сумму; прогресс пересчитывается.
ТС-12	Расчёт регулярного накопления	В уроке задать цель 70, накоплено 20, срок 5 недель.	Правильный ответ: 10 монет в неделю.
ТС-13	Ошибочный ответ в уроке	Выбрать заведомо неверный вариант.	Показывается объяснение без потери предыдущего прогресса; доступна повторная попытка/продолжение.
ТС-14	Награда за урок	Завершить урок и забрать награду; открыть его снова.	Первое получение добавляет награду; claimedLessonRewards не позволяет начислить её повторно.
ТС-15	Последовательность уроков	Завершить доступный урок.	Следующий урок/глава открывается согласно порядку content.ts.
ТС-16	История операций	Выполнить несколько покупок/пополнений.	Операции отображаются в истории; объём сохранённой истории не превышает 100 записей.

ТС-17	Офлайн-режим	Отключить Wi-Fi и мобильную сеть, перезапустить приложение.	Основной игровой цикл, уроки, покупки и локальный прогресс работают без сети.
ТС-18	Сохранение прогресса	Пройти урок, изменить баланс/цель, полностью закрыть приложение и открыть снова.	Состояние восстанавливается из AsyncStorage без сброса прогресса.
ТС-19	Удаление/сброс данных	В разделе взрослого/настройки подтвердить удаление профиля.	Ключи finny-profile и finny-game очищены; приложение возвращается к первому запуску.
ТС-20	Экран 360 dp и системный шрифт	Проверить на ширине 360 dp и при увеличенном системном масштабе текста.	Нет обрезки ключевого текста/кнопок, основные действия доступны.
ТС-21	Минимальные разрешения	Проверить сведения о приложении/AndroidManifest после установки.	Нет запросов камеры, микрофона, геолокации, контактов и Bluetooth без функциональной необходимости.
ТС-22	Стабильность и отклик	Пройти обязательный сценарий без перезапуска.	Нет падений/тупиков; локальные действия получают визуальный отклик примерно до 1 секунды.

10.2. Краткий отчёт о проверке на физическом устройстве

Техническое задание требует финального прогона как минимум на одном физическом Android-устройстве с оперативной памятью от 3 ГБ. В предоставленных материалах модель устройства, версия Android, объём ОЗУ и журнал фактического прогона не зафиксированы, поэтому результаты нельзя корректно обозначить как «пройдено». Ниже оставлен минимальный протокол, который необходимо заполнить после установки релизного APK.

Параметр	Результат / что зафиксировать
Модель устройства	Не указано — заполнить после прогона
Версия Android	Не указано — Android 8.0 или выше
Оперативная память	Не указано — требуется ≥ 3 ГБ

Версия APK / номер сборки	Не указано — взять из финальной release/preview сборки
Время запуска до стартового/главного экрана	Измерить; целевое значение ≤ 5 секунд
Отклик локальных действий	Измерить; целевое значение ≤ 1 секунды
Обязательный сценарий	Пройти TC-01...TC-19 без падений и тупиков
Офлайн-работа	Проверить TC-17
Сохранение после перезапуска	Проверить TC-18
Итог	Заполнить: «пройдено» только после фактического прогона

11. Известные ограничения и план развития

11.1. Текущие ограничения прототипа

- Приложение полностью локальное: нет облачной синхронизации, переноса прогресса между устройствами и резервного восстановления после удаления данных.
- Отдельный демонстрационный режим с тестовым профилем и мгновенным переходом через несколько игровых периодов в доступной документации не подтверждён.
- Долгосрочное развитие питомца документировано через xp/level и текущую сытость; соответствие требованию о минимум трёх стадиях роста, зависящих от серии финансовых решений, требует доработки или явной демонстрации.
- Сравнение утверждённого бюджетного плана с фактическими расходами за период не описано как отдельный пользовательский экран.
- Снятие средств с финансовой цели с отдельным подтверждением и пересчётом срока достижения в документации не подтверждено.
- Барьер входа в раздел взрослого и фактический сценарий удаления/сброса локального профиля требуют проверки на релизной сборке.
- Доступность не прошла документированный аудит: нужно проверить экранный диктор, системное увеличение шрифта, размеры touch-target и режим уменьшения анимаций.
- Автоматизированные тесты в предоставленной документации не перечислены; ключевая логика должна быть покрыта тестами либо подтверждена полным ручным прогоном.
- Фактический отчёт о тестировании на физическом устройстве и точные параметры устройства пока не зафиксированы.
- Версия/номер финальной Android-сборки, package name и полный реестр лицензий визуальных/звуковых ассетов должны быть сверены перед публикацией.

11.2. План дальнейшего развития

Этап	Работы
Приоритет 0 — до финальной сдачи	Провести полный прогон APK на физическом Android-устройстве; зафиксировать модель, Android, ОЗУ, время запуска и результат тест-кейсов. Проверить package name/version и AndroidManifest.
Приоритет 0 — до финальной сдачи	Проверить/добавить взрослый барьер, локальный сброс и удаление профиля; добавить демонстрационный тестовый профиль, если его нет.
Приоритет 0 — до финальной сдачи	Сверить все изображения, шрифты и звуки с источниками и лицензиями; оставить в репозитории файл THIRD_PARTY_NOTICES или аналогичный реестр.
Приоритет 1	Развить модель питомца до минимум трёх устойчивых стадий роста, рассчитываемых по совокупности решений: закрытие обязательных расходов, соблюдение бюджета, регулярность накоплений.
Приоритет 1	Добавить экран «план / факт» по игровому периоду и объяснение отклонений простыми словами.
Приоритет 1	Добавить подтверждаемое снятие средств с цели и пересчёт прогресса/срока достижения.
Приоритет 1	Провести аудит доступности: масштаб текста, TalkBack, accessibilityRole/Label, reduced motion, контраст и touch-target.
Приоритет 2	Расширять учебный контент через независимые конфигурации content.ts/lessonFlows.ts: новые темы, задания и повторение без переработки базовой архитектуры.
Приоритет 2	Поддержать планшеты и, при необходимости, iOS; проверить адаптивность и альбомную ориентацию.
Приоритет 2	При появлении облачной синхронизации — проектировать её как опциональную функцию с отдельным согласием законного представителя и минимизацией данных.

12. Сторонние библиотеки и лицензии

12.1. Библиотеки и пакеты

Ниже перечислены ключевые зависимости, зафиксированные в документе. Для финальной сдачи полный перечень следует сверить с package.json/package-lock.json. Большинство используемых библиотек распространяется по разрешительной лицензии

MIT; TypeScript — Apache License 2.0. EAS CLI является инструментом сборки и включает пакеты под разными лицензиями; сам пакет eas-cli распространяется по лицензии MIT; лицензии отдельных вспомогательных компонентов сборки EAS перед публикацией стоит сверить отдельно (не входят в состав самого APK и на лицензионную чистоту приложения не влияют).

Компонент	Лицензия	Назначение / примечание
Expo SDK 57.0.26	MIT	Фреймворк/SDK приложения
React 19.2.3	MIT	UI-библиотека
React DOM 19.2.3	MIT	Зависимость React/Expo для web-совместимости
React Native 0.86.3	MIT	Нативный слой приложения
Expo Router 57.0.24	MIT	Файловая маршрутизация
Zustand 5.0.15	MIT	Управление состоянием
@react-native-async-storage/async-storage 2.2.0	MIT	Локальное постоянное хранение
react-native-reanimated 4.5.1	MIT	Анимации
react-native-worklets 0.10.1	MIT	Worklets/выполнение анимационной логики
react-native-gesture-handler 2.32.0	MIT	Жесты и интерактивность
react-native-screens 4.26.0	MIT	Нативные контейнеры экранов
react-native-safe-area-context 5.7.0	MIT	Безопасные области экрана
react-native-svg 15.15.4	MIT	SVG-графика
react-native-keyboard-controller 1.21.9	MIT	Управление экранной клавиатурой
TypeScript 6.0.3	Apache-2.0	Инструмент разработки; не является самостоятельной runtime-библиотекой APK
EAS CLI 24.8.0	MIT (сам eas-cli; используется только как инструмент сборки, в APK не входит)	Инструмент облачной/локальной сборки; не поставляется как часть пользовательского приложения

12.2. Шрифты, изображения, звуки — источники и лицензии

Тип ресурса	Источник / размещение	Лицензия и действие перед сдачей
Шрифты интерфейса	В предоставленной документации отдельный пользовательский шрифт не указан.	Перед сдачей проверить assets/fonts и theme.ts. Если используются только системные шрифты платформы, отдельный файл шрифта в составе приложения отсутствует.

Персонаж Финни и его состояния	assets/ — персонажи по стадиям и цветам.	В предоставленных материалах внешний правообладатель/лицензия не указаны. Если графика создана командой, зафиксировать «собственная графика команды»; если использовался внешний источник — указать автора, ссылку и лицензию.
Иллюстрации еды и гардероба	assets/food, assets/wardrobe или соответствующие каталоги ресурсов.	Требуется финальная сверка происхождения файлов и права на распространение.
Иллюстрации уроков и элементы интерфейса	assets/learning и UI-ассеты.	Требуется финальная сверка происхождения файлов и права на распространение.
Звуки	В архитектуре предусмотрена настройка soundEnabled, но перечень аудиофайлов не приведён.	Если звуковые файлы присутствуют, добавить имя файла, источник/автора и лицензию. Если аудиофайлов нет — зафиксировать «не используются».

Рекомендуемый формат реестра для репозитория: файл THIRD_PARTY_NOTICES.md с колонками «файл/пакет — версия — автор/источник — лицензия — ссылка на текст лицензии». Это позволит однозначно подтвердить право использования всех сторонних компонентов и медиаресурсов.